

Практическая работа 9 по Информационным технологиям

Язык ассемблера на примере RARS. Макропроцессор

Для выполнения этой практической работы вам потребуются материалы [лекции 8 «Ассемблер RISC-V. Эмулятор RARS. Макропроцессор»](#).

Также можно воспользоваться описанием макропроцессора RARS из его Help.

В таблице 1 приведены некоторые системные вызовы из справки RARS.

Таблица 1. Некоторые системные вызовы RARS

Функция	Тип a7	Параметры	Возвращаемое значение
вывод целого	1	a0 = целое	
вывод вещественного	2	fa0 = вещественное	
вывод вещественного двойной длины	3	fa0 = двойное вещественное	
вывод строки	4	a0 = адрес ASCIIZ-строки	
ввод целого	5		a0 — введённое целое (word)
ввод вещественного	6		fa0 — введённое вещественное (float)
ввод двойного вещественного	7		fa0 — введенное двойное вещественное (double)
ввод строки	8	a0 = адрес памяти для размещения строки a1 = размер буфера	Если строка короче буфера, все её символы записываются в буфер (включая '\n', если он был), после чего туда дописывается нулевой байт. Если строка не короче буфера, в него записывается первые a1-1 символов, а в последний оставшийся байт записывается 0. Таким образом формируется ASCIIZ-строка.
выделение памяти в Куче (sbrk)	9	a0 = размер требуемой памяти в байтах	a0 — адрес требуемой памяти
останов (exit)	10		
вывод символа	11	a0 = символ	Некоторые управляющие символы, например, перевод строки (код 10) тоже имеет смысл выводить.
ввод символа	12		a0 — считанный символ (традиционно в RARS работает отвратительно, не рекомендуется 😞)
системное время (time)	30		Время выдаётся в миллисекундах с начала мира (полночь на 1 января 1970 года), так что помещается как длинное целое в два слова a1 = старшее слово a0 = младшее слово
приостановка выполнения (sleep)	32	a0 = время простоя в миллисекундах.	
вывод шестнадцатеричного	34	a0 = целое	
вывод двоичного	35	a0 = целое	
вывод беззнакового	36	a0 = целое	

При написании макросов хорошим тоном считается сначала составить *подпрограмму*, а её вызов уже «обернуть» в макрос. В этом случае макроподстановка растиражирует только преамбулу и вызов подпрограммы, а содержательный текст будет оттранслирован единожды в её составе. Например:

```
.text
_input: # a0 – message / a0 – input value
        li      a7 4
        ecall
        li      a7 5
        ecall
        ret

.macro   input    %msg %reg
        la      a0 %msg
        jal     _input
        mv      %reg a0
.end_macro

_print: # a0 – message, a1 – number
        li      a7 4
        ecall
        mv      a0 a1
        li      a7 1
        ecall
        li      a0 10
        li      a7 11
        ecall
        ret

.macro   print    %msg %reg
        la      a0 %msg
        mv      a1 %reg
        jal     _print
.end_macro

.data
msg1:    .asciz  "First number: "
msg2:    .asciz  "Second number: "
res1:    .asciz  "Result 1: "
res2:    .asciz  "Result 2: "

.text
.globl   main
main:
        input    msg1 t0
        input    msg2 t1
        print    res1 t0
        print    res2 t1
```

Задание 1. Ввод с подсказкой (Input Prompt)

Написать два макроса:

а) **input** строка-подсказка регистр-приёмник, который выводит на экран строку-подсказку, а затем помещает в регистр-приёмник введённое целочисленное значение

б) `print` строка-подсказка `регистр-источник`, который выводит на экран строку-подсказку, а затем — целочисленное значение `регистра-источника` и перевод строки.

Пример тестирующей программы:

```
.text
    input    "First input: " s0
    input    "Second input: " s1
    print    "First result: " s0
    print    "Second result: " s1
    li       a7 10
    ecall
```

Input:

First input: 2
Second input: 3

Output:

First result: 2
Second result: 3

Задание 2. Строковые макросы (StrMacro)

Написать комплект макросов для работы со строками. Параметры макросов — адреса (метки) ASCII-строк в памяти. Результаты работы сохраняются в регистрах `a0` и `a1`:

Макрокоманда	Параметр 1	Параметр 2	Описание	→a0	→a1
<code>strlen</code>	Адрес		Определение длины строки	Длина строки	
<code>strcpy</code>	Приёмник	Источник	Копирует строку с адреса «источник» по адресу «приёмник»	Приёмник	Длина строки-приёмника
<code>strcat</code>	Приёмник	Источник	Копирует строку с адреса «источник» в конец строки по адресу «приёмник»	Приёмник	Длина строки-приёмника

- Копирование должно происходить «справа налево», чтобы строку с адресом `A` можно было скопировать на адрес `A + k`. Например, если метка `A` — это адрес, а `B` — это адрес + 1, то `strcpy B A` должно приводить к удвоению первого символа `A`.
- Аналогично, `strcat STR STR` должен приводить к удвоению строки `STR`
- Разрешается модифицировать регистры типа `t*`
- Все три подпрограммы — «небезопасные»: `strcpy A B` из предыдущих примеров может привести к бесконечному циклу или к заполнению строки последним символом, а отсутствие нулевого байта — к выходу за пределы памяти.

Пример тестирующей программы:

```
.globl main
.data
```

```

src:      .asciz  "Source"
dst:      .asciz  "Destination"
fin:      .asciz  "Destination+Source"
.text
main:
    strcpy  fin src
    strcat  fin dst
    la      a0 fin
    li      a7 4
    ecalls
    strlen  fin
    li      a7 1
    ecalls

```

Input:

<пустой ввод>

Output:

SourceDestination17

Задание 3. Многочлен (Polydouble)

Написать макрос **POLY** **Массив** **Регистр** **D-регистр**, который будет вычислять значение многочлена $A_0 + A_1x + A_2x^2 + \dots + A_nx^n$ (с двойной точностью).

- **Массив** – это адрес массива **A** коэффициентов многочлена, начиная с A_0
- **Регистр** – это целочисленный регистр, содержащий порядок многочлена **n** ($0 \leq n \leq 100$)
- **D-регистр** – это вещественный регистр двойной точности, содержащий значение переменной **x**

Результат вычислений помещается в D-регистр. Пример вызывающей программы:

```

.data
_x:      .double 1.1
_p345:   .double 3, 4, 5
.text
.globl  main
main:
    fld    ft0 _x t0
    li     t0 2
    POLY   _p345 t0 ft0
    fmv.d  fa0 ft0
    li     a7 3
    ecalls

```

Input:

<пустой ввод>

Output:

13.4500000000000001

Задание 4. Векторная операция (VectorOp)

Написать макрос `VectorOp Массив_А Массив_Б длина операция`, который поэлементно заполняет `Массив_А` результатами выражения `операция(Массив_А[i], Массив_Б[i])`.

- `Массив_А` и `Массив_Б` — адреса начала массивов; размер ячейки массива — 32 бита;
- `длина` — натуральное число ≤ 1000 ;
- `операция` — адрес функции, которая в `a0` возвращает результат некоторой операции над `a0` и `a1`

Пример тестирующей программы:

```
.data
A:      .space 4000
B:      .space 4000
.text
        VectorOp      A A 5 input
        VectorOp      B B 5 input
        VectorOp      A B 5 sum
        VectorOp      A A 5 print
        li             a7 10
        ecall
sum:     add           a0 a0 a1
        ret
input:   li           a7 5
        ecall
        ret
print:   li           a7 1
        ecall
        ret
```

Input:

1
2
3
4
5
6
7
8
9
10

Output:

79111315

Контрольные вопросы

1. *Что такое макрос?*
2. *Что такое макроопределение?*
3. *Что такое макроподстановка?*
4. *Что такое макрорасширение?*
5. *Что такое директива?*